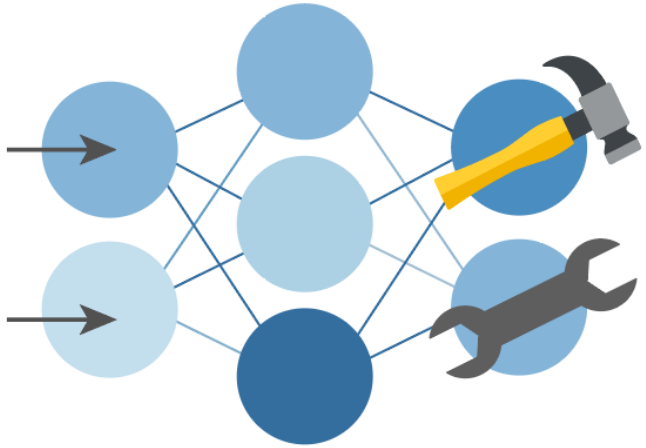
Python + Agents



🛠️ Feb 24: Building your first agent in Python
🧠 Feb 25: Adding context and memory to agents
📊 Feb 26: Monitoring and evaluating agents
🔀 Mar 3: Building your first AI-driven workflows
🕸️ Mar 4: Orchestrating advanced multi-agent workflows
🙋🏽‍♀️ Mar 5: Adding a human-in-the-loop to workflows

🔗 **Register at aka.ms/PythonAgents/series**

Microsoft

Python + Agents

Orchestrating advanced multi-agent workflows

aka.ms/pythonagents/slides/advancedworkflows

Pamela Fox

Python Cloud Advocate

www.pamelafox.org

Today we'll cover...

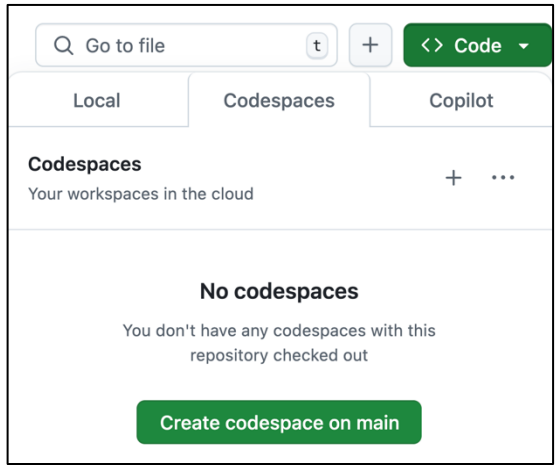
- Concurrent workflows with fan-out and fan-in edges
- Aggregation patterns: summary, ranking, voting, extraction
- Conditional routing with concurrent execution
- Built-in orchestrations:
  - ConcurrentBuilder
  - MagenticBuilder for dynamic planning
  - HandoffBuilder for dynamic routing
- E2E application with workflows

Want to follow along?

1. Open this GitHub repository:

aka.ms/python-agentframework-demos

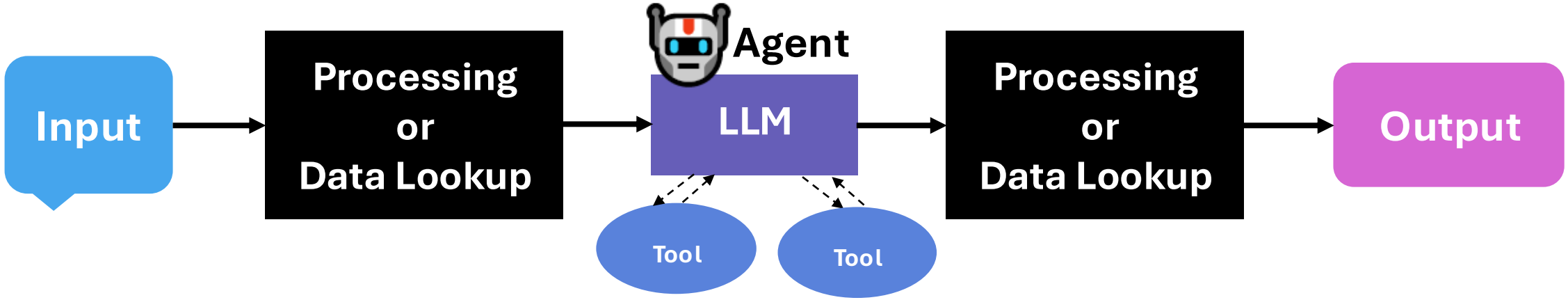
2. Use "Code" button to create a GitHub Codespace:



3. Wait a few minutes for Codespace to start up ⏱️
4. Run `git pull` to ensure you have the latest changes

Recap: What's an agentic workflow?

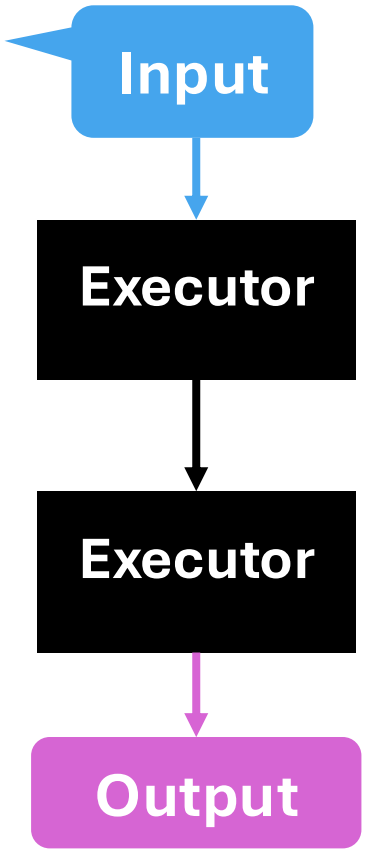
An agentic workflow is a flow that involves an agent at some point, typically to handle decision making or answer synthesis.



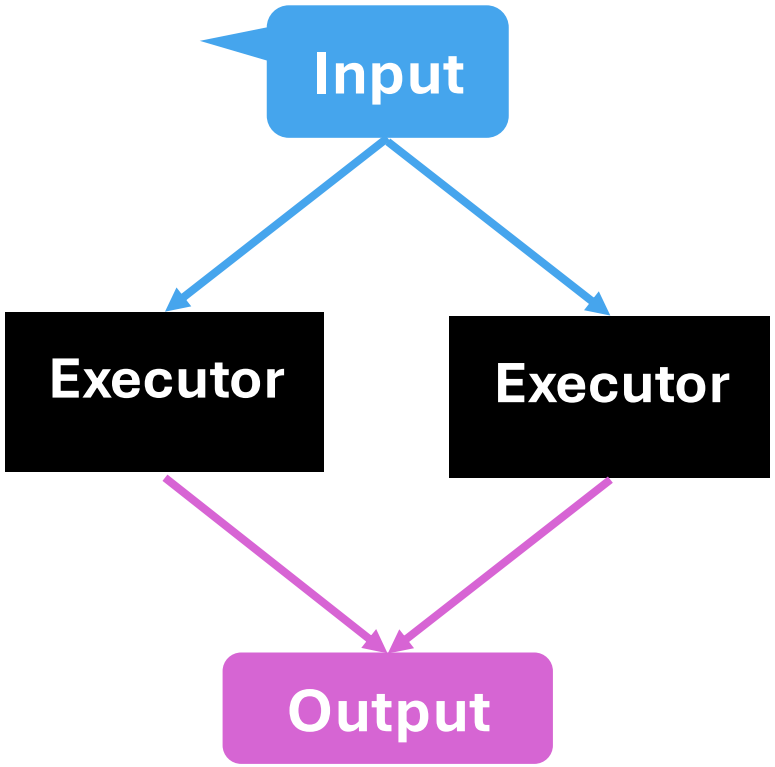
In agent-framework, a workflow is a graph with `Executor` nodes and edges between:

Concurrent execution

Sequential execution vs. Concurrent execution



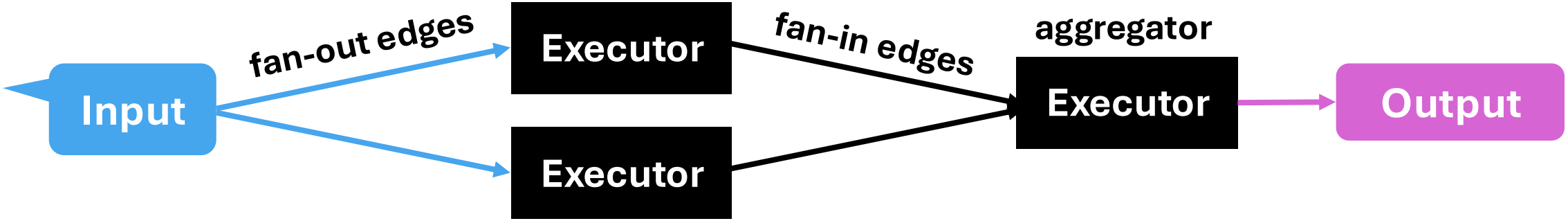
Only one executor runs at a given time.



Executors run in parallel
and outputs are aggregated.

Anatomy of a concurrent workflow

This is a common topology, where inputs **fan-out** and outputs **fan-in** to **aggregator**:



Also possible:

- workflows with dynamic fan-out (optional executors)
- workflows without an explicit aggregator

agent-framework

Using fan-in/fan-out edges in workflows

Use `WorkflowBuilder` with `add_fan_out_edges` and `add_fan_in_edges`:

```
workflow = (WorkflowBuilder(
    name="FanOutFanInEdges",
    description="Explicit fan-out/fan-in using edge groups.",
    start_executor=dispatcher,
   output_executors=[aggregator])
  .add_fan_out_edges(dispatcher, [researcher, marketer, legal])
  .add_fan_in_edges([researcher, marketer, legal], aggregator)
  .build())
```

Full example: [workflow_fan_out_fan_in_edges.py]

agent-framework

Aggregating results from fan-out edges

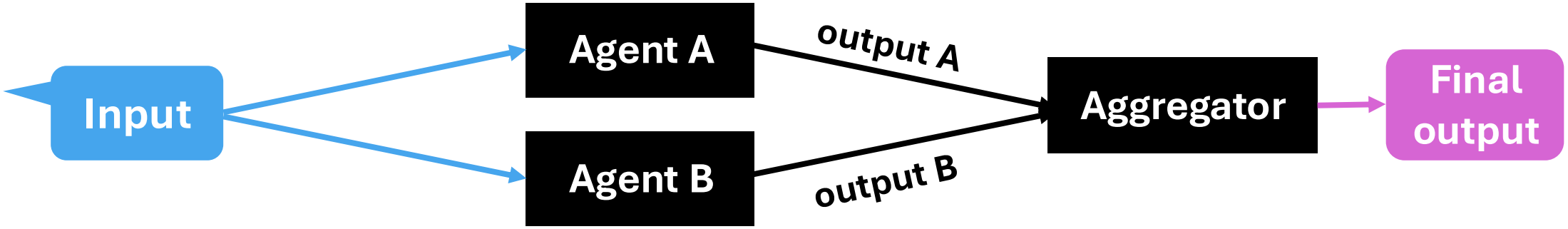
```python
class AggregateInsights(Executor):

  @handler
  async def aggregate(
    self,
    results: list[AgentExecutorResponse],
    ctx: WorkflowContext[Never, str]):
    for result in results:
      # Process result.executor_id and result.agent_response.text
    await ctx.yield_output(consolidated)
```

Full example: [workflow_fan_out_fan_in_edges.py]

Aggregation

Aggregation patterns for fan-in agents



Pattern	Aggregator implementation	Final output	When to use?
Concatenation	String operations or template	Formatted str	You control the layout exactly
Output synthesis	LLM call to summarize outputs	Natural prose	You want a human-readable summary
Structured extraction	LLM call with structured outputs	Pydantic model	Downstream code needs typed data
Score & rank	LLM-as-a-judge call to score outputs	Ranked list	Generate N options, pick the best
Majority vote	Count most common output label	Winner label	Multiple judges classify same input

agent-framework

Aggregation with LLM summary

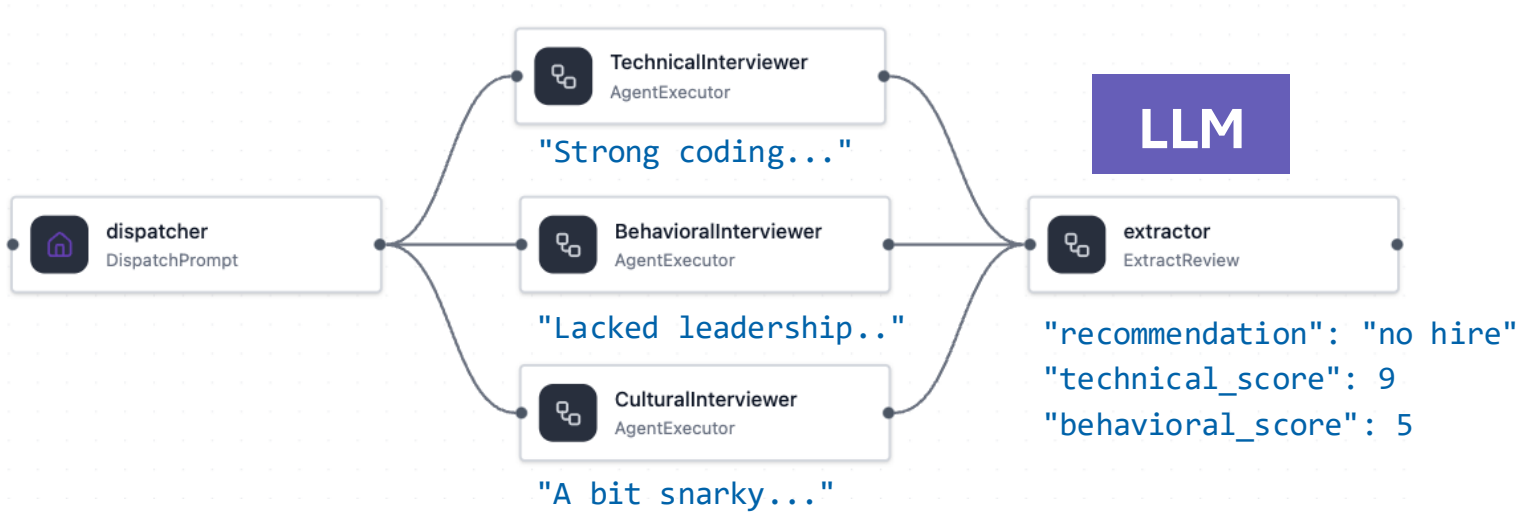
The aggregator node uses an LLM call to aggregate the combined outputs from agents.

```
class SummarizerExecutor(Executor):
  def __init__(self, client: OpenAIChatClient, id: str):
    super().__init__(id=id)
    self.agent = Agent(client=client, name=id, instructions="Summarize...")
  @handler
  async def run(self, results, ctx):
    sections = [f"[{r.executor_id}]\n{r.agent_response.text}"
                for r in results]
    response = await self.agent.run("\n".join(sections))
    await ctx.yield_output(response.text)
```

Full example: [workflow_aggregator_summary.py]

agent-framework

Aggregation with structured extraction




```python
class CandidateReview(BaseModel):
  technical_score: int
  technical_reason: str
  behavioral_score: int
  behavioral_reason: str
  recommendation: Literal[
   "strong hire", "no hire",
   "hire with reservations"]
```

```python
class ExtractReview(Executor):
  @handler
  async def extract(self, results, ctx):
    combined = "\n".join(r.agent_response.text for r in results)
    messages = [Message(role="system", text="Review candidate."),
                Message(role="user", text=combined)]
    response = await self._client.get_response(messages,
                                options={"response_format": CandidateReview})
    await ctx.yield_output(response.value)
```

Full example: workflow_aggregator_structured.py

agent-framework

Aggregation with LLM-ranking

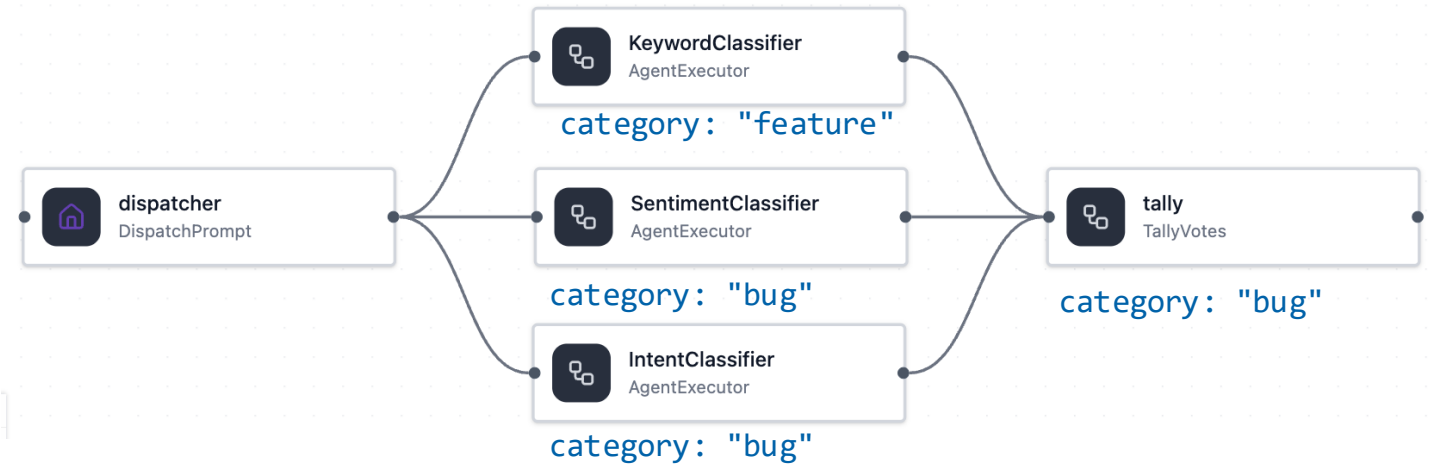
The aggregator node uses an LLM call to rank the outputs from the agents.

```
class RankerExecutor(Executor):
  @handler
  async def run(self, results, ctx):
    slogans = [f"{r.executor_id}: \"{r.agent_response.text}\"" for r in results]
    messages = [Message(role="system", text=("Score each slogan 1-10 with a reason")),
                Message(role="user", text="Candidate slogans:\n" + "\n".join(slogans))]
    response = await self._client.get_response(messages,
                                               options={"response_format": RankedSlogans})
    await ctx.yield_output(response.value)
```

Full example: [workflow_aggregator_ranked.py]

agent-framework

Aggregation with majority vote



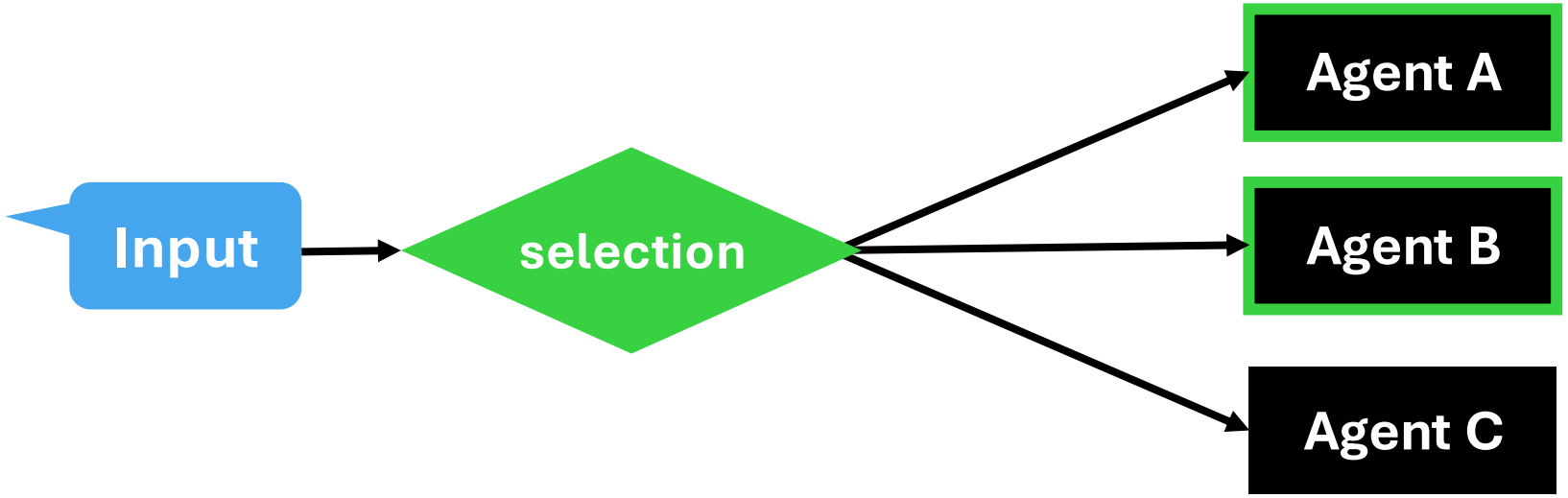
Each agent outputs their vote, and the aggregator calculates the most popular choice.

```
class TallyVotes(Executor):
  @handler
  async def tally(self, results: list[AgentExecutorResponse],
                                ctx: WorkflowContext[Never, str]):
    votes = []
    for result in results:
      classification: Classification= result.agent_response.value
      votes.append(classification.category.value)
    winner, count = Counter(votes).most_common(1)[0]
    await ctx.yield_output(winner)
```

Full example: [workflow_aggregator_voting.py]

Conditional routing + concurrency

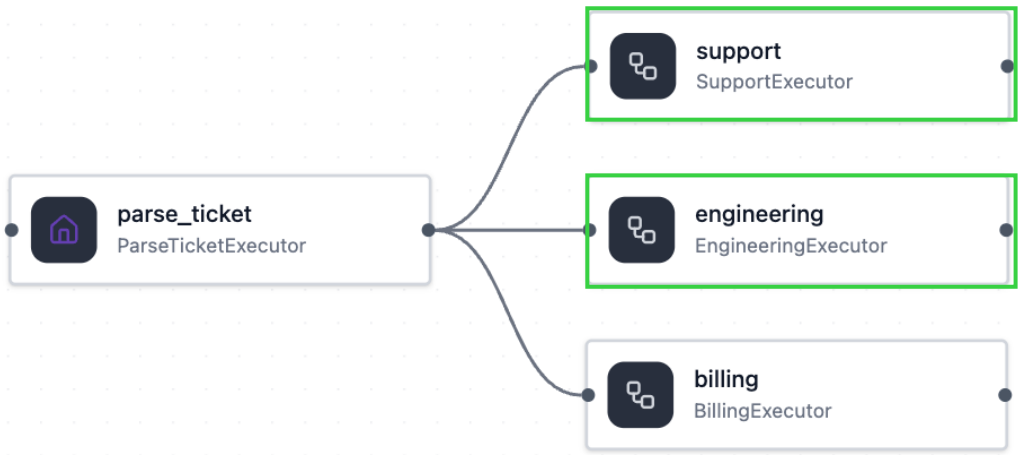
Conditional routing with concurrent execution



The input is sent to any number of concurrent executors based off a selection function.

agent-framework

Workflows with multi-selection edge groups



Based off the `selection_func`, the workflow sends the input to the selected executors for concurrent processing.

```
workflow = (WorkflowBuilder(name="MultiSelectionEdgeGroup",
                            start_executor=parse_ticket)
            .add_multi_selection_edge_group(
                parse_ticket,
                [support, engineering, billing],
                selection_func=select_targets)
            .build())
```

Full example: [workflow_multi_selection_edge_group.py]

Built-in workflow orchestrations

https://learn.microsoft.com/agent-framework/workflows/orchestrations/

ConcurrentBuilder

https://learn.microsoft.com/agent-framework/workflows/orchestrations/concurrent

agent-framework

Using the built-in ConcurrentBuilder workflow

```
from agent_framework.orchestrations import ConcurrentBuilder

workflow = ConcurrentBuilder(
            participants=[researcher, marketer, legal]).build()
```

The workflow assumes that participants accept and emit conversation-shaped data, so executors must be `Agent`s or custom `Executor`s that process conversations.

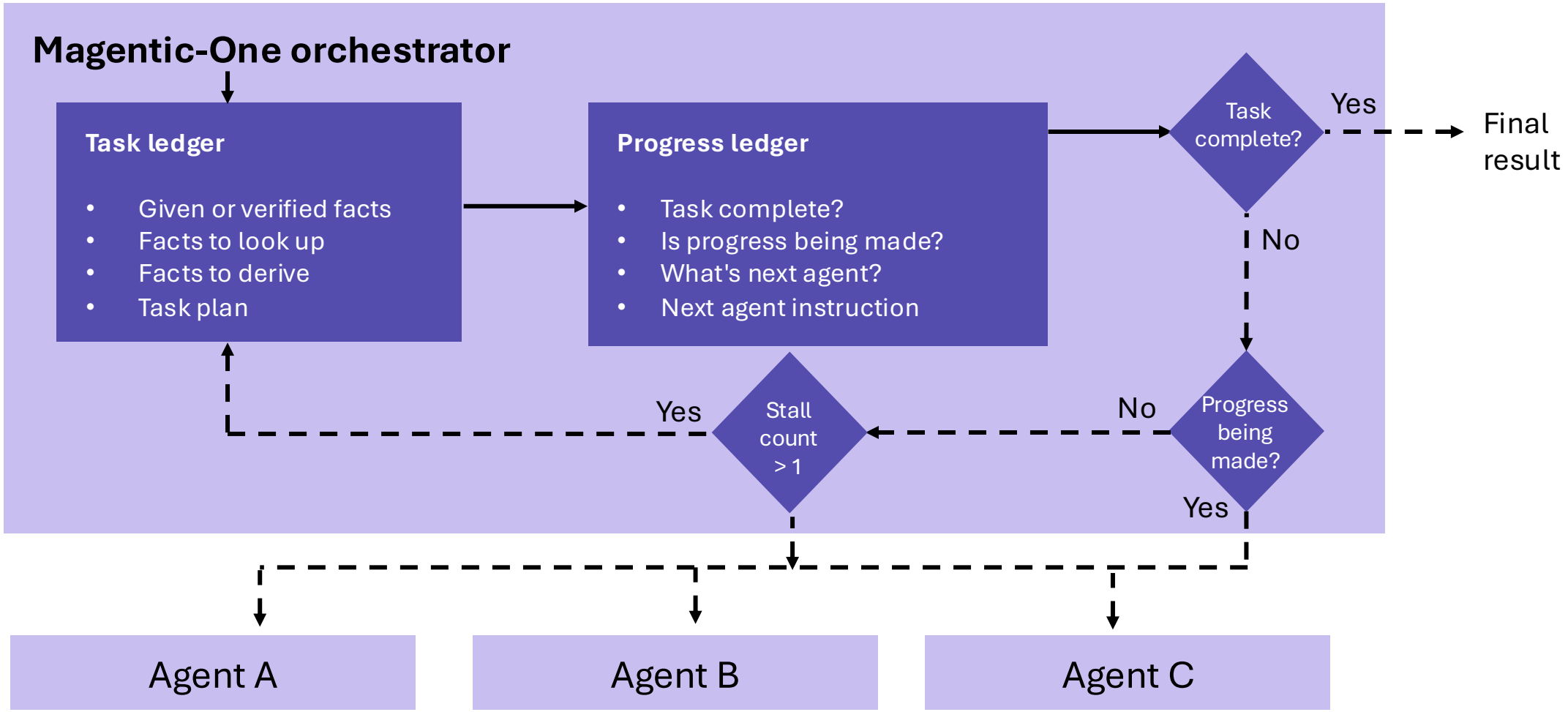
The extra start/end nodes normalize input and output to `list[Message]`.

Full example: [workflow_agents_concurrent.py]

MagenticBuilder for dynamic planning

https://learn.microsoft.com/agent-framework/workflows/orchestrations/magentic

Magentic-One workflow orchestration



https://arxiv.org/abs/2411.04468

Magentic-One: Initial plan development

The orchestrator uses this prompt to plan:

Specialist agents

```
To address this request we have assembled the following team:

- local_agent: A local assistant that can suggest local activities or
places to visit.

- language_agent: A helpful assistant that can provide language tips
for a given destination.

- travel_summary_agent: A helpful assistant that can summarize the
travel plan.

Based on the team composition, and known and unknown facts, please
devise a short bullet-point plan for addressing the original request.
Remember, there is no requirement to involve all team members. A team
member's particular expertise may not be needed for this task.
```

Magentic-One: Progress ledger updates

The orchestrator assesses progress after every turn, picks the next agent and tells it what to do.

Has the task been done?

Are agents repeating themselves?

Are we moving forward?

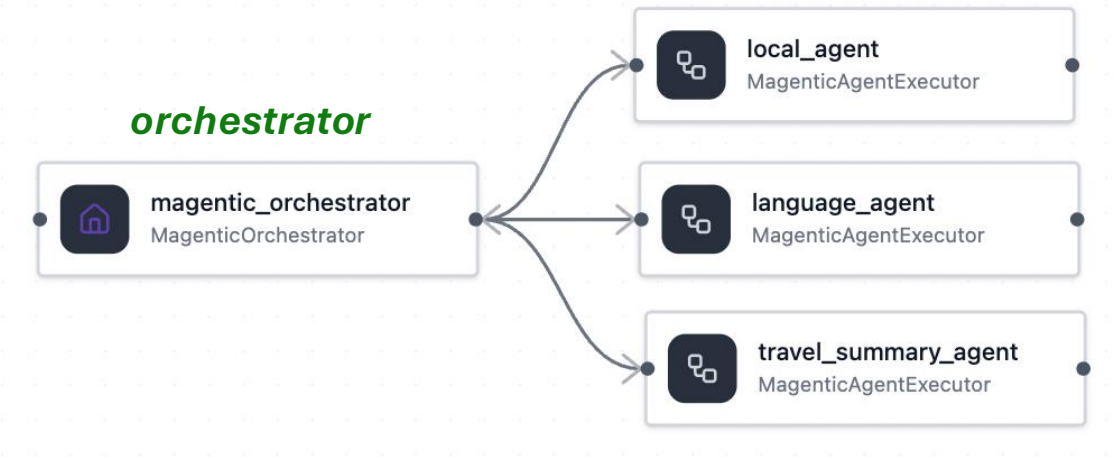
Who should go next?

What should they do?

```
{"is_request_satisfied": {
    "answer": "no", "reason": "..."
  },
  "is_in_loop": {
    "answer": "no", "reason": "..."
  },
  "is_progress_being_made": {
    "answer": "yes", "reason": "...",
  },
  "next_speaker": {
    "answer": "travel_summary_agent", "reason": "...",
  },
  "instruction_or_question": {
    "answer": "Summarize the proposal", "reason": "...",
}}
```

agent-framework

Using the built-in MagenticBuilder workflow

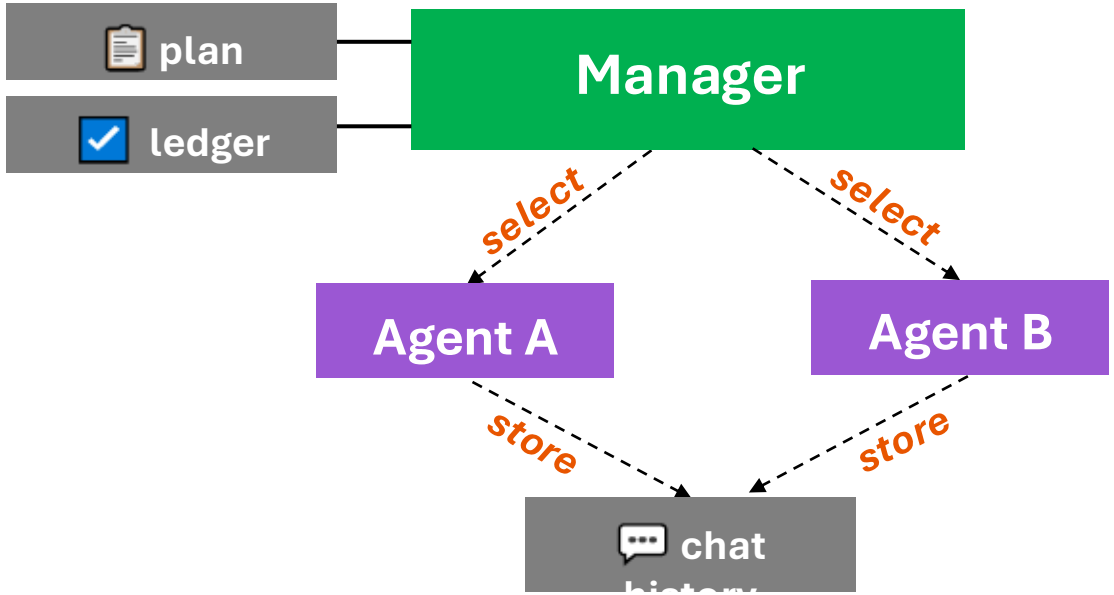


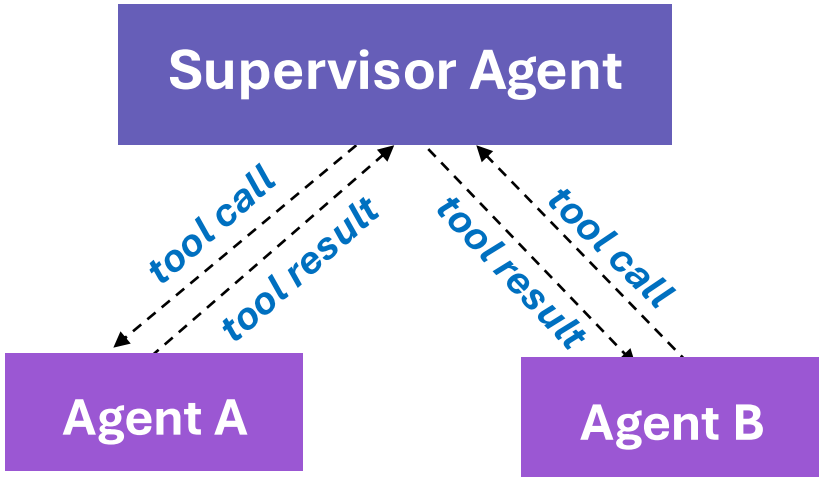
The `MagenticOrchestrator` node contains a `StandardMagenticManager`, which wraps the provided `manager_agent`, and plans and delegates tasks.

```
magentic_workflow = MagenticBuilder(
  participants=[local_agent, language_agent, travel_summary_agent],
  manager_agent=manager_agent,
  max_round_count=10,
  max_stall_count=1,
  max_reset_count=1,
  ).build()
```

Full example: [workflow_magenticone.py]

Magentic orchestration vs. Agent-as-tools





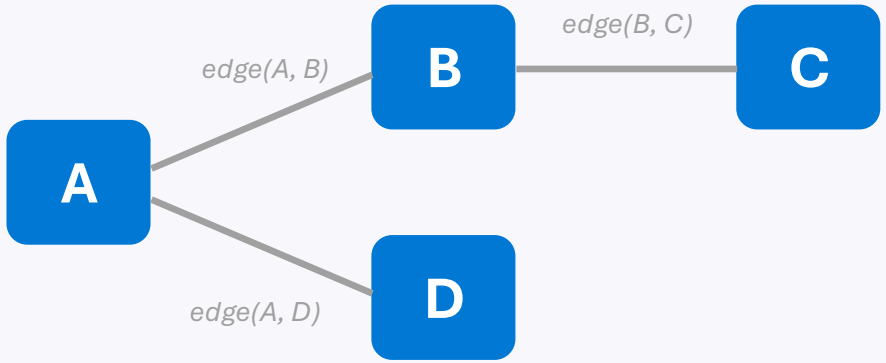
Planning	Explicit (plan with progress ledger)	Implicit (reactive tool call decisions)
Loop detection	Progress ledger checks is_in_loop	No built-in loop detection
Agent context	Full shared chat history	Isolated per-call context
Token cost	Higher (plan + ledger+ shared history)	Lower

Dynamic routing with HandoffBuilder

From fixed graphs to dynamic routing

FIXED GRAPH

Developer defines edges. LLM picks a path.



- All edges declared at build time
- LLM selects which branch, not which agent
- Topology is static — predictable and testable

WorkflowBuilder · SequentialBuilder · ConcurrentBuilder

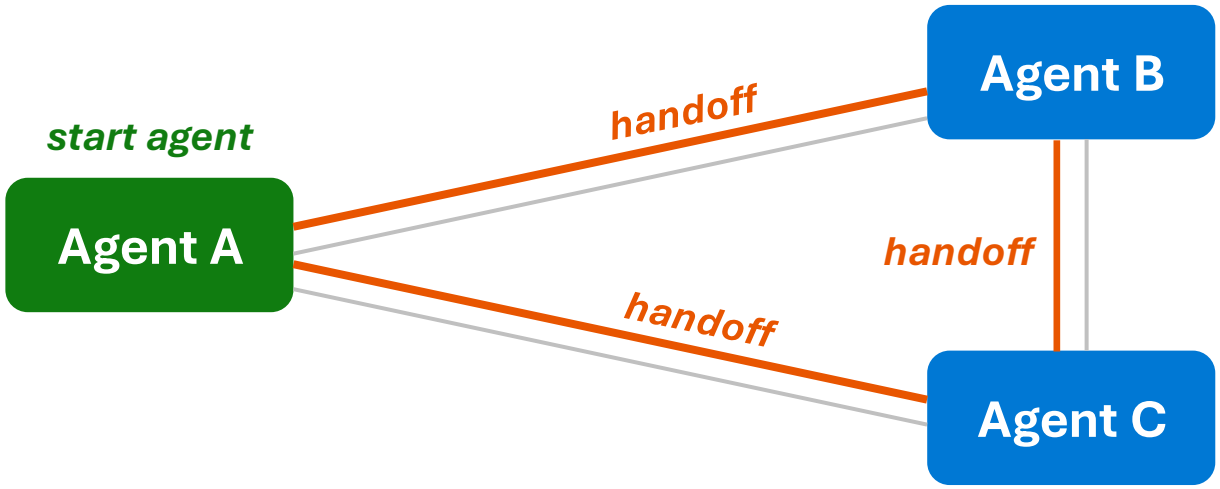
DYNAMIC ROUTING

Agents decide the next step at runtime.

- No predefined edges between agents
- Routing emerges from conversation state
- Topology is dynamic — flexible and adaptive

HandoffBuilder

Handoff orchestration



The start agent just kicks off the flow.

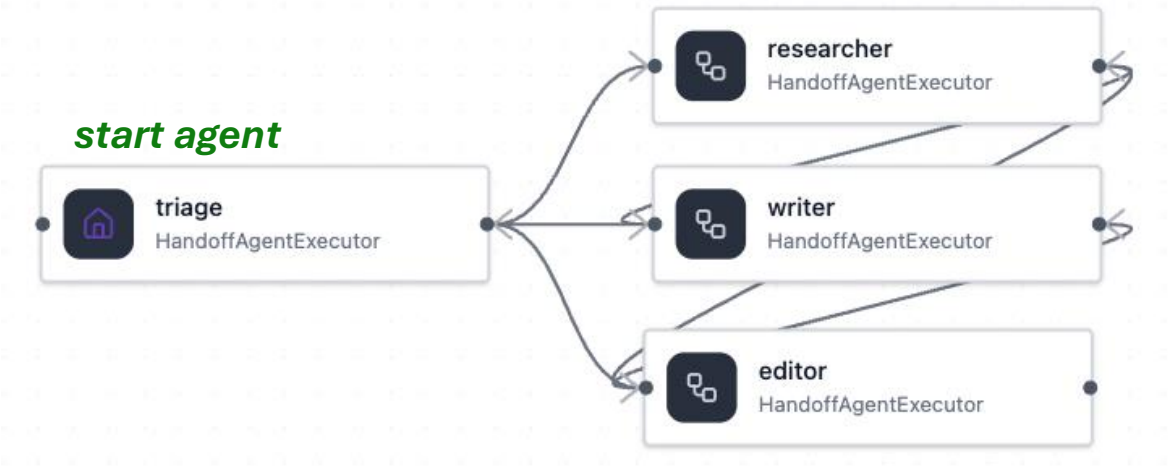
Any agent can hand off control to any other agent at runtime.

The receiving agent fully owns the conversation until it hands back.

No edges are defined up front – routing emerges from conversation state.

agent-framework

Using the built-in HandoffBuilder workflow



By default, all agents in a handoff orchestration can handoff to every other agent.

In autonomous mode, they will never ask the user for input.

```
workflow = (HandoffBuilder(
    name="content_pipeline",
    participants=[triage, researcher, writer, editor],
    termination_condition=lambda conversation: (len(conversation) > 0
        and conversation[-1].text.strip().startswith("FINAL:")))
  .with_start_agent(triage)
  .with_autonomous_mode()
  .build())
```

Full example: [workflow_handoffbuilder.py]

agent-framework

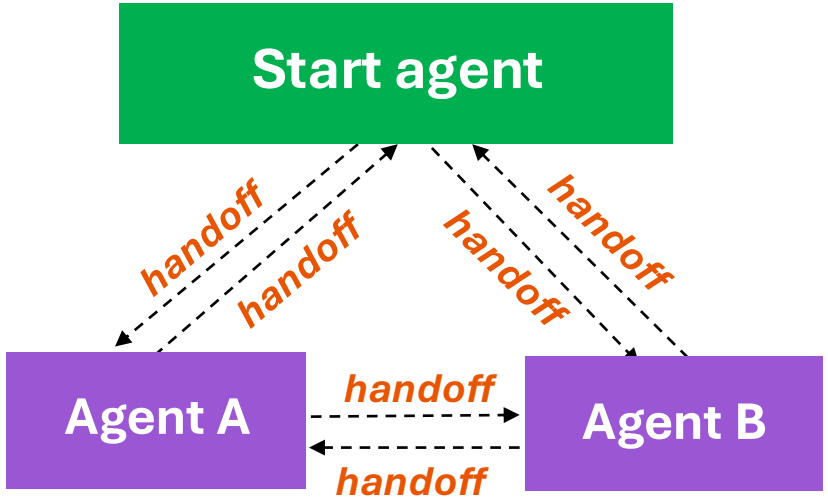
HandoffBuilder with handoff rules

```
workflow = (HandoffBuilder(
  participants=[triage_agent, order_agent, return_agent, refund_agent],
  termination_condition=lambda conversation: (
    len(conversation) > 0 and "goodbye" in conversation[-1].text.lower()))
    .with_start_agent(triage_agent)
    .add_handoff(triage_agent, [order_agent, return_agent])
    .add_handoff(return_agent, [refund_agent, triage_agent])
    .add_handoff(order_agent, [triage_agent])
    .add_handoff(refund_agent, [triage_agent])
    .with_autonomous_mode().build())
```

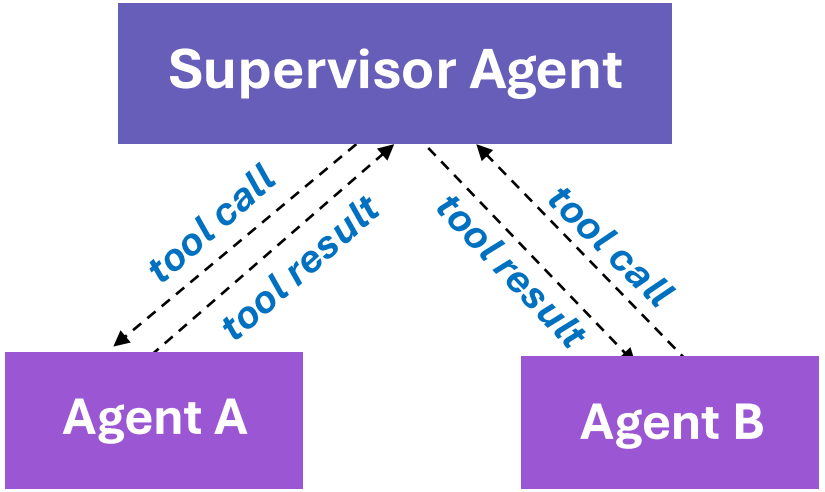
Full example: [workflow_handoffbuilder_rules.py]

Handoff orchestration vs. Agent-as-tools

Delegated ownership



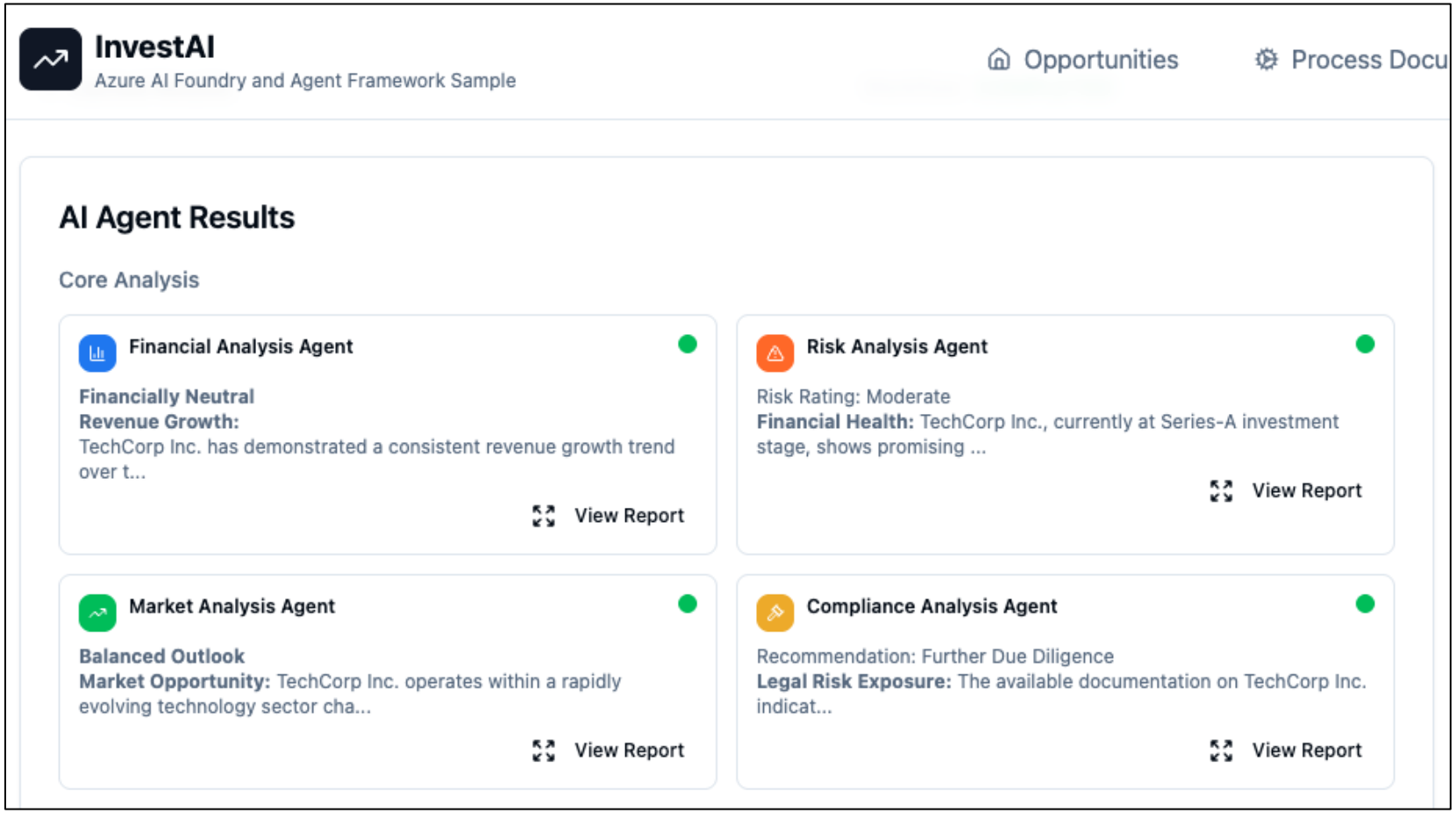
Centralized ownership



Control Flow	Active agent transfers control to another agent (peer-to-peer).	Supervisor calls sub-agent tools, control returns to supervisor.
Task Ownership	Ownership moves to receiving agent.	Supervisor retains ownership end-to-end.
Context Management	Conversation context is handed over.	Supervisor selects what context each tool gets.

End-to-end application

Agentic AI investment analysis



Agent Framework*
FastAPI
Azure OpenAI
Azure Cosmos DB
Azure Blob Storage

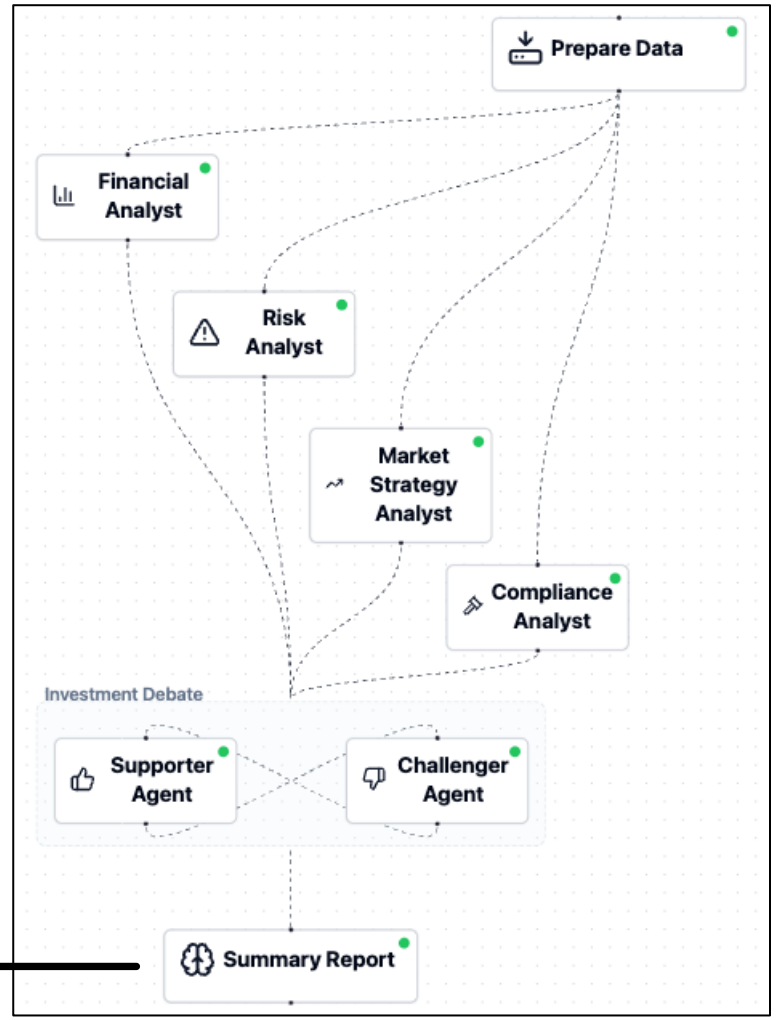
**Uses an older MAF version, upgrades are pending.*

https://github.com/Azure-Samples/Agentic-AI-Investment-Analysis-Sample

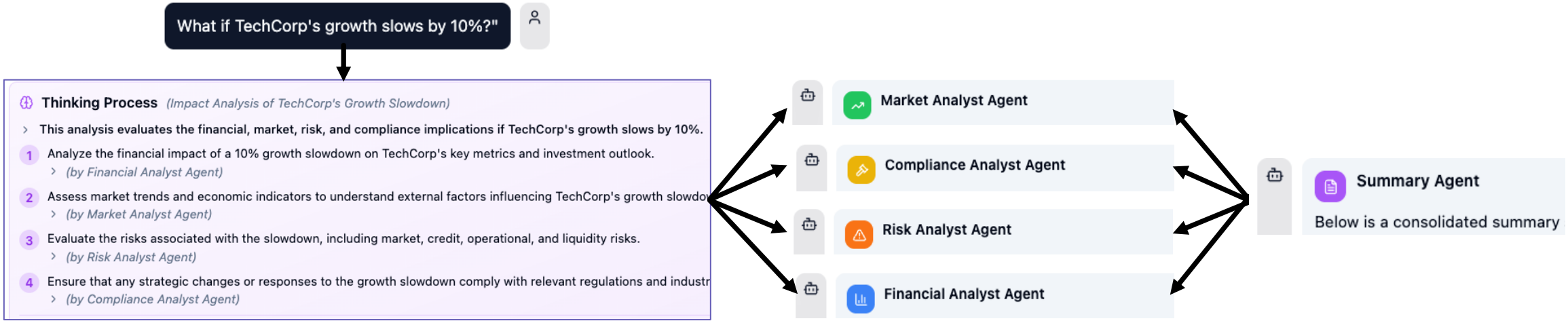
Investment workflow with fan-out/fan-in

```
workflow = WorkflowBuilder()
  .set_start_executor(data_prep_executor)
  .add_fan_out_edges(data_prep_executor,
     [financial_analyst, risk_analyst,
      market_analyst, compliance_analyst])
  .add_fan_in_edges(
     [financial_analyst, risk_analyst,
      market_analyst, compliance_analyst],
     analysis_aggregator)
  .add_edge(analysis_aggregator, debate_executor)
  .add_edge(debate_executor, summary_report_generator)
  .build()
```

[Full file: investment_workflow.py](investment_workflow.py)



What-if workflow with fan-in/fan-out




```
workflow = WorkflowBuilder()
  .set_start_executor(planner_agent)
  .add_fan_out_edges(planner_agent,
     [financial_analyst_agent, risk_analyst_agent,
      market_analyst_agent, compliance_analyst_agent])
  .add_fan_in_edges(
     [financial_analyst_agent, risk_analyst_agent,
      market_analyst_agent, compliance_analyst_agent],
     summarizer_agent)
  .build()
```

Full file: what_if_workflow.py

Next steps

Register:
https://aka.ms/PythonAgents/series

Watch past recordings:
aka.ms/pythonagents/resources

Join office hours after each session in Discord:
aka.ms/pythonai/oh

🛠️ Feb 24: Building your first agent in Python
🧠 Feb 25: Adding context and memory to agents
📊 Feb 26: Monitoring and evaluating agents
🔀 Mar 3: Building your first AI-driven workflows
🕸️ Mar 4: Orchestrating advanced multi-agent workflows
🙋🏽‍♀️ Mar 5: Adding a human-in-the-loop to workflows